A machine-checked proof that the maximum number of stable matchings of order five is sixteen

Jiarui Xu*

August 31, 2026

Abstract

Let $f(n)$ be the maximum number of stable matchings of a stable-marriage instance with n men and n women; determining f is a problem of Knuth (1976) and Open Problem #1 of Gusfield and Irving (1989). The value $f(5) = 16$ was announced by Eilers (2022, OEIS A357269) on the strength of an unpublished constraint-programming search that produced no checkable artifact. We give the first machine-checkable proof. A Lean 4 development defines the problem in ∼40 lines, proves—with no `sorry` and only the three standard Mathlib axioms—the theorem

$$\texttt{f5\_eq\_16\_of\_unsat}: \Big(\forall\, \rho \in P_{120},\ \neg\mathrm{Sat}(\Phi_\rho)\Big) \longrightarrow \Big(\forall I,\ \mathrm{WF}(I) \to \mathrm{sc}(I) \le 16\Big) \wedge \Big(\exists I,\ \mathrm{WF}(I) \wedge \mathrm{sc}(I) = 16\Big),$$

where the 120 propositional formulas Φ_ρ are themselves defined in Lean and printed verbatim to DIMACS by a 30-line exporter. Each Φ_ρ is refuted by kissat, each proof is validated by drat-trim, and each resulting LRAT certificate is checked end-to-end by the formally verified checker cake_lpr: 120/120 verified, about two hours on a laptop. The architecture follows the empty-hexagon development of Subercaseaux et al. (ITP 2024). As by-products we independently confirm Eilers' companion counts (OEIS A344669): there are exactly 507,254,400 maximal profiles of order 5 (176,130 up to symmetry), obtained by enumerating all 4,227,120 canonical solutions.

1 Introduction

A stable-marriage instance of order n equips n men and n women each with a strict preference order over the opposite side; a perfect matching is *stable* if no man and woman prefer each other to their assigned partners. Gale and Shapley proved stable matchings always exist. Let $f(n)$ denote the maximum number of stable matchings over instances of order n. Knuth posed the behaviour of f as a research problem in 1976 [1]; Gusfield and Irving made it Open Problem #1 of their monograph [2]. Asymptotically $2.28^n \le f(n) \le 3.55^n$ [3, 4, 5], and exactly (OEIS A357269 [6]):

$$f(1), \ldots, f(5) = 1,\ 2,\ 3,\ 10,\ 16.$$

*jxucoder@gmail.com. The formal development, computations and drafting were carried out with the assistance of Claude (Anthropic); every numerical claim comes from executed, logged runs in the accompanying repository, and every formal claim is checked by the Lean kernel.

The last and hardest value, $f(5) = 16$, was announced by Dan Eilers in September 2022, computed with a MiniZinc constraint-programming model. The result is trustworthy—we confirm it here by independent means—but it had, until now, no proof object: constraint solvers do not emit certificates, the search is not reproducible from any published artifact, and the paper announced alongside the OEIS entry has not appeared. For contrast, the best human-provable upper bound at $n = 5$ is $3.55^5 \approx 555$.

Contributions.

1. **A machine-checked proof of** $f(5) = 16$ in the architecture of [7]: a Lean 4 development (zero `sorry`s; axioms `propext`, `Classical.choice`, `Quot.sound`) reduces the theorem to the unsatisfiability of 120 concrete CNF formulas that are *defined inside Lean* and printed verbatim for the solvers; the 120 refutations are checked by the formally verified proof checker cake_lpr [9]. The kernel-checked part comprises a symmetry reduction, a case-covering lemma, and the encoding-faithfulness theorem; the lower bound is a kernel-computed evaluation of an explicit witness.

2. **Independent confirmations** of every unreplicated number in this corner of the OEIS: $f(5) = 16$ itself, and the maximal-profile counts A344669$(3, 4, 5) = 1092,\ 144,\ 507{,}254{,}400$ together with Eilers' reduced count 176,130, via exhaustive enumeration with a different toolchain.

3. **A complete verification artifact**: a repository with a step-by-step third-party verification guide (`VERIFYING.md`), pinned toolchain versions and hashes, a differential-testing harness binding the formal definitions to an independently validated reference implementation, and continuous integration that, on pushes to `main`, pull requests and manual dispatch, rebuilds the development from source (with the standard Mathlib olean cache) and checks the axiom base.

2 The Lean development

2.1 The review surface

The claim of a formal theorem is only as strong as the definitions in its statement, so we display them in full. An instance is a pair of 5×5 rank tables; everything is built from `List Nat`:

```
structure Inst where
  mrank : List (List Nat)   -- mrank[m][w] = position of w in m's order, 0 best
  wrank : List (List Nat)   -- symmetric

def isRankRow (r : List Nat) : Bool :=
  r.length = 5 && (List.range 5).all fun v => r.contains v

def WF (I : Inst) : Bool :=
  I.mrank.length = 5 && I.wrank.length = 5 &&
  I.mrank.all isRankRow && I.wrank.all isRankRow

def get2 (t : List (List Nat)) (i j : Nat) : Nat := (t.getD i []).getD j 0
```

```
def isStable (I : Inst) (mu : List Nat) : Bool :=
  (List.range 5).all fun m => (List.range 5).all fun w =>
    let wm := mu.getD m 0        -- m's partner
    let mw := idxOf w mu         -- w's partner
    (w = wm) || !(get2 I.mrank m w < get2 I.mrank m wm &&
                  get2 I.wrank w m < get2 I.wrank w mw)

def stableCount (I : Inst) : Nat :=
  (([0,1,2,3,4] : List Nat).permutations.filter (isStable I)).length
```

(plus the seven-line first-order recursion `idxOf`). This, together with the statement below, is the entire human-review surface; nothing else needs to be read to trust the theorem:

```
theorem f5_eq_16_of_unsat
    (H : forall row in perms120, not (Satisfiable (cubeCNF row))) :
    (forall I : Inst, WF I = true -> stableCount I <= 16) /\
    (exists I : Inst, WF I = true /\ stableCount I = 16)
```

where `perms120` and `cubeCNF` are likewise concrete Lean definitions (Section 3), so the single hypothesis `H` is exactly what the verified certificate checker discharges.

2.2 Upper bound: symmetry, covering, faithfulness

The kernel-checked chain has three stages.

Symmetry (`reduce_man0'`, `Symmetry.lean`, 416 lines): relabelling women by man 0's rank row σ fixes man 0's row to the identity while preserving well-formedness and sc. The proof constructs the relabelling explicitly — `relabel` composes every man's row with σ^{-1} and permutes the women's rows — together with the induced bijection $\mu \mapsto \sigma^{-1} \circ \mu$ on matchings, and carries stability across it pointwise. The workhorses are four small lemmas about the first-order `idxOf` recursion (`idxOf_lt_length`, `getD_idxOf`, `idxOf_getD`, `idxOf_map`) which together say that on lists that are permutations of $[0, \ldots, 4]$, indexing and inversion behave like the group theory one would do on paper. No `Equiv.Perm` or finite-group machinery is used; staying first-order over lists keeps the definitions kernel-computable and the faithfulness proofs syntactic.

Covering (`cube_covering`): for well-formed I, man 1's rank row lies in the 120-element list $P_{120} =$ `permutations` $[0, \ldots, 4]$; this is `rankRow_perm` plus membership transport, and it is what makes the 120 cubes exhaustive by construction rather than by convention.

Faithfulness (`cube_faithful'`, `Faithfulness.lean`, 429 lines): for each $\rho \in P_{120}$, any well-formed I with man 0 identity, man 1 row ρ, and $\mathrm{sc}(I) \geq 17$ yields an assignment satisfying the CNF Φ_ρ of Section 3. Chaining the three: if every Φ_ρ is unsatisfiable then $\mathrm{sc}(I) \leq 16$ for all well-formed I.

2.3 Lower bound: a kernel-computed witness

An explicit witness instance (found by SAT in 0.1s) is embedded as a literal:

```
def witnessI : Inst :=
  ⟨[[2,0,1,4,3], [1,3,4,2,0], [2,4,3,0,1], [4,1,0,2,3], [0,2,3,4,1]],
   [[1,3,2,0,4], [4,1,0,3,2], [2,0,1,4,3], [1,2,3,4,0], [0,4,3,1,2]]⟩
```

and sc(`witnessI`) = 16 is proved by kernel computation. One subtlety is worth recording for practitioners: Mathlib's `List.permutations` is defined by well-founded recursion, which the kernel cannot unfold, so `decide` fails on `stableCount` as stated. The count is transported across the Mathlib equivalence `List.permutations_perm_permutations'` to the structurally recursive `permutations'` and evaluated there — a two-line fix once diagnosed, invisible in the final statement. A standalone variant (`Witness.lean`) proves the same count from first principles using only `propext`. We deliberately avoid `native_decide` throughout: every computation in the trust base is kernel-reduced.

Main theorem. `f5_eq_16_of_unsat` combines both directions; `#print axioms` reports exactly `[propext, Classical.choice, Quot.sound]`, re-checked — together with a no-`sorry` sweep and a rebuild of the development from source with the standard Mathlib olean cache — by continuous integration on pushes to `main`, pull requests and manual dispatch.

3 The certificates

Encoding. The variables of Φ_ρ come in two blocks. *Preference variables*: for each side $s \in \{\text{men}, \text{women}\}$, person $i < 5$ and pair $a < b < 5$, a variable $p^s_{i,a,b}$ meaning "person i prefers a to b" ($2 \cdot 5 \cdot 10 = 100$ variables); a literal for "prefers a to b" with $a > b$ is the negation. *Selector variables*: $y_{t,\mu}$ for $t < 17$ and μ ranging over the concrete list P_{120} of matchings ($17 \cdot 120 = 2040$ variables; 2140 in all). The clause families:

- *transitivity*: for every person and ordered triple, forbid both preference 3-cycles ($2 \cdot 5 \cdot \binom{5}{3} \cdot 2$ clauses) — with totality this makes each person's preference relation a strict linear order;
- *cube units*: unit clauses pinning man 0's row to the identity and man 1's row to ρ;
- *slot nonemptiness*: $\bigvee_\mu y_{t,\mu}$ for each $t < 17$;
- *slot ordering*: $\neg y_{t,\mu} \vee \neg y_{t+1,\mu'}$ for $\mu' \leq \mu$ in the fixed enumeration order of P_{120} — consecutive slots strictly increase, killing the 17! slot symmetry and forcing seventeen *distinct* selections;
- *non-blocking*: $\neg y_{t,\mu} \vee \neg p^m_{m,w,\mu(m)} \vee \neg p^w_{w,m,\mu^{-1}(w)}$ for every slot, matching and non-matched pair (m, w) — a selected matching admits no blocking pair.

Together: 157,197 clauses. The encoding is deliberately *naive*: no cardinality networks, no auxiliary Tseitin variables, because every clause family must be discharged by hand in the faithfulness proof. Satisfiability of Φ_ρ says precisely "some instance in cube ρ has 17 pairwise-distinct stable matchings".

Faithfulness, in detail. Given well-formed I in cube ρ with $\text{sc}(I) \geq 17$, the assignment τ sets $p^s_{i,a,b}$ by comparing the actual ranks in I, and sets $y_{t,\mu}$ true iff μ is the t-th stable matching of I in the enumeration order of P_{120} (the first seventeen indices exist because $\text{sc}(I) \geq 17$). Six lemmas then discharge the six families (`trans_sat`, `man0_sat`, `man1_sat`, `nonempty_sat`, `ordering_sat`, `block_sat`); the last is the delicate one, unfolding the Boolean `isStable` of the selected matching at the offending pair. The composition is `cube_faithful'`. As a positive control against vacuity, the 16-slot variant of the cube containing the witness is satisfiable, and the decoded model recounts to a valid 16-matching instance.

Pipeline. The formulas are printed from the Lean definitions by a 30-line exporter. Each cube is refuted by kissat 4.0.4 (hardest cube 115s — the one containing the extremal instances), each DRAT proof is validated and elaborated to LRAT by drat-trim, and each LRAT is checked by cake_lpr, whose soundness is itself machine-checked down to machine code [9]. Result: **120/120** `s VERIFIED UNSAT`, about two hours end-to-end on an Apple M4 Max (8-way parallel). Outside the Lean kernel the trust base is cake_lpr and the 30-line printer, and nothing else — the solvers are checked, not trusted.

4 Cross-validation

Four independent anchors tie the formal definitions to reality: (1) a reference counter validated against the extremal Latin instances of orders $3, 4, 5, 6$ recorded in OEIS A351413 (3/10/9/48, all reproduced); (2) a differential test: on 300 random instances the Lean definitions and the reference counter agree exactly; (3) the witness chain: SAT model $\to$ reference recount $\to$ Lean kernel count, all 16; (4) enumeration: blocking-clause enumeration over all 120 cubes yields exactly 4,227,120 canonical (man-0 = id) profiles with 16 stable matchings; multiplying by the 5! woman-relabellings gives 507,254,400 = A344669(5), and dividing by the 4! man-relabellings gives 176,130, Eilers' reduced count — both previously unreplicated. The same runs confirm A344669(3) = 1092 (full brute force over all 46,656 profiles) and A344669(4) = 144 (six canonical solutions).

Before adopting the present architecture we also refuted "≥ 17" monolithically three times (different encodings and solver configurations, including a pure-RUP kissat –plain proof of 2.1GB verified by drat-trim, and a Mathlib `lrat_proof` kernel import of the analogous n=4 certificate, which reproves $f(4) = 10$ inside Lean in ten seconds).

5 Development statistics and lessons

file	lines	theorems	role
`Faithful.lean`	94	3	problem definitions, covering
`Symmetry.lean`	416	35	woman-relabelling reduction
`Encoding.lean`	109	1	CNF as a Lean value; DIMACS printer
`Faithfulness.lean`	429	26	assignment τ; six family lemmas
`Bridge.lean`, `Lower.lean`	82	6	composition; witness
total ($f(5)$)	1,130	71	

The development builds in about twenty minutes (Mathlib cached); the kernel evaluation of the witness count takes seconds. Some transferable lessons:

- *Single source of truth for the encoding.* The classic gap in certified-SAT papers is between the CNF the paper describes and the CNF the solver saw. Here the CNF is a Lean value and the DIMACS files are printed from it by a 30-line exporter, so the faithfulness theorem is about the very formulas that were refuted. The exporter is in the trust base; it is small enough to read in one sitting.

- *Stay first-order, stay computable.* All objects are lists of naturals; stability is a `Bool`-valued program. This makes every quantity in the development kernel-computable (`decide`

works), keeps proofs syntactic, and avoids simultaneously fighting the mathlib abstraction stack and the SAT bridge.

- *Certified results as test oracles.* Before formalization, the counting function was validated against OEIS ground truth and by differential testing; after formalization, the certified theorem became the oracle for the (unverified) $f(6)$ search machinery of the companion paper — certification pays forward.
- *Positive controls.* Every UNSAT campaign ran alongside a SAT control (the 16-slot cube of the witness) to guard against vacuous refutations of over-constrained encodings — the SAT analogue of testing that your test suite can fail.

On AI assistance. The development — problem selection, Lean proofs, encodings, search code, and this paper — was produced by the author working with Claude (Anthropic) in an agentic coding setup, over roughly three days. The formal artifact is unaffected in kind by this provenance: the theorem statement is forty lines of displayed Lean, the axioms are printed by the kernel, and the certificates are checked by a verified checker. We note it because the workflow — an AI system iterating against a proof assistant, with the human directing scope, priorities and publication decisions — was essential to the three-day cost, and because problem-selection due diligence (checking OEIS entries, problem pages and prior announcements) consumed a substantial fraction of the effort, which we suspect generalizes.

6 Related work

Our architecture — verified encoding in Lean, certificates checked by cake_lpr, unsatisfiability entering the main theorem as an explicit hypothesis — is that of the empty hexagon number verification [7]. The end-to-end Keller-conjecture verification [8] has since internalized even the checker into Lean by reflection; migrating our certificate step onto such a checker (or LRAT-Catcher [13]) is mechanical future work. Other certified computational values include $R(4,5) = 25$ [10], Schur number five, the packing chromatic number of the square grid, and $K_8(4,2) = 23$ [11]. On the stable-matching side the computational lineage is Eilers' ($f(4)$, $f(5)$, the profile counts) building on Thurber [3] and ultimately Knuth [1]; the asymptotic bounds are [4, 5].

7 Conclusion

$f(5) = 16$ now has a proof whose every step is checked either by the Lean kernel or by a formally verified certificate checker, with a forty-line human-review surface. The companion paper [12] takes the next step: it establishes the previously open value $f(6) = 48$ by reducing instance space ($\sim 10^{28}$) to a space of rotation schedules, covering the canonical schedules by cubes of a Lean-defined formula and refuting all 318,736 certified cubes with cake_lpr-checked certificates; an exhaustive enumeration of the 2.7×10^{10} schedule nodes, validated against the certified theorem proved here, corroborates the result. The reduction lemmas of [12] are themselves formalized in Lean (a further 5,230 lines and 243 theorems in the eleven files from `SixBridge.lean` through `SchedCNF6.lean`, the last defining the order-6 formula; zero `sorry`s, in the same repository). The order-6 faithfulness theorem — that every well-formed instance with at least 49 stable matchings satisfies some certified cube's formula — is proved as well (a further 19 files, 4,761

lines and 333 theorems), so that $f(6) = 48$ is likewise a single Lean theorem, `f6_eq_48_of_unsat`, with 318,736 cake_lpr-checked certificates as its only hypothesis.

Artifact. Repository with code, proofs, logs, verification guide (`VERIFYING.md`) and CI; certificate archive with DOI to accompany the final version. Reproduction cost: ∼20 minutes for the Lean development, ∼2–3 hours for the certificates.

Acknowledgments. The priority for $f(5) = 16$ and for the A344669 counts belongs to Dan Eilers; this work confirms his announcements and supplies the proof objects. We thank the OEIS, and the authors of kissat, drat-trim and cake_lpr.

References

[1] D. E. Knuth, *Mariages stables*, Presses de l'Université de Montréal, 1976.

[2] D. Gusfield and R. W. Irving, *The Stable Marriage Problem: Structure and Algorithms*, MIT Press, 1989.

[3] E. G. Thurber, Concerning the maximum number of stable matchings in the stable marriage problem, *Discrete Math.* 248 (2002), 195–219.

[4] A. R. Karlin, S. Oveis Gharan and R. Weber, A simply exponential upper bound on the maximum number of stable matchings, *STOC 2018*.

[5] C. Palmer and D. Pálvölgyi, At most 3.55^n stable matchings, *FOCS 2021*.

[6] OEIS, sequences A357269, A357271, A344669, A351413, `https://oeis.org`.

[7] B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro and M. J. H. Heule, Formal verification of the empty hexagon number, *ITP 2024*; arXiv:2403.17370.

[8] J. Gallicchio, C. Codel, J. Avigad and M. J. H. Heule, An end-to-end verification of Keller's conjecture, *ITP 2026*.

[9] Y. K. Tan, M. J. H. Heule and M. O. Myreen, cake_lpr: verified propagation redundancy checking in CakeML, *TACAS 2021*.

[10] T. Gauthier and C. E. Brown, A formal proof of $R(4, 5) = 25$, arXiv:2404.01761, 2024.

[11] A Lean-certified proof of $K_8(4, 2) = 23$, arXiv:2606.16688, 2026.

[12] J. Xu, The maximum number of stable matchings of order six is forty-eight, companion manuscript, 2026.

[13] LRAT-Catcher: importing SAT solver certificates into Lean 4 by reflection, arXiv:2607.00815, 2026.